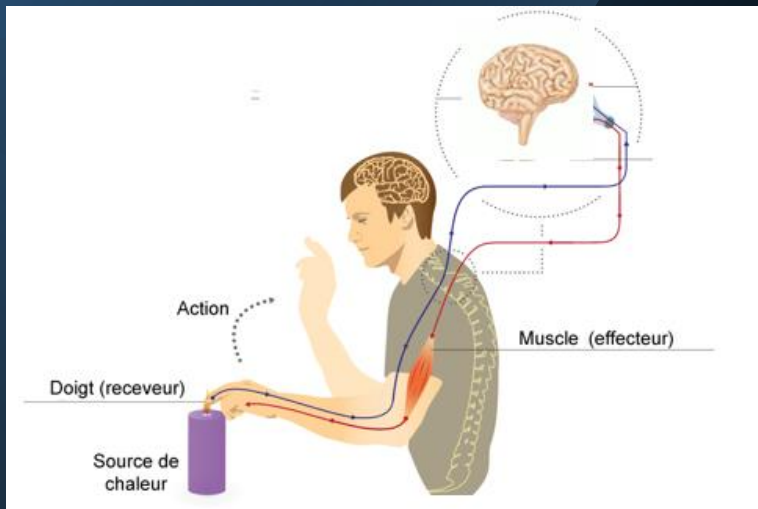


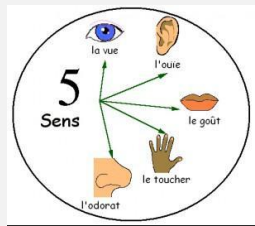
Codification et représentation des nombres

De quoi est
composé un
système
automatisé?



- **Pourquoi si tu touches quelque chose de chaud, tu retireras ta main?**
 - Parce qu'il y a au moins 200 000 récepteurs de détection de la température sur la peau.
 - Les nerfs sensitifs transportent l'information de la peau vers le cerveau.
 - L'information est traitée par le cerveau.
 - Le cerveau donne des ordres aux muscles à travers les nerfs moteurs pour retirer ta main.

De quoi est composé un système automatisé?



Les **récepteurs sensoriels** (oeil, oreille, peau, nez et langue) permettent au corps d'un individu d'être informé sur l'état de son environnement

→ Les capteurs



Les muscles → Les actionneurs



Le cerveau → Le microprocesseur

De quoi est composé un système automatisé?



Exemple Pratique

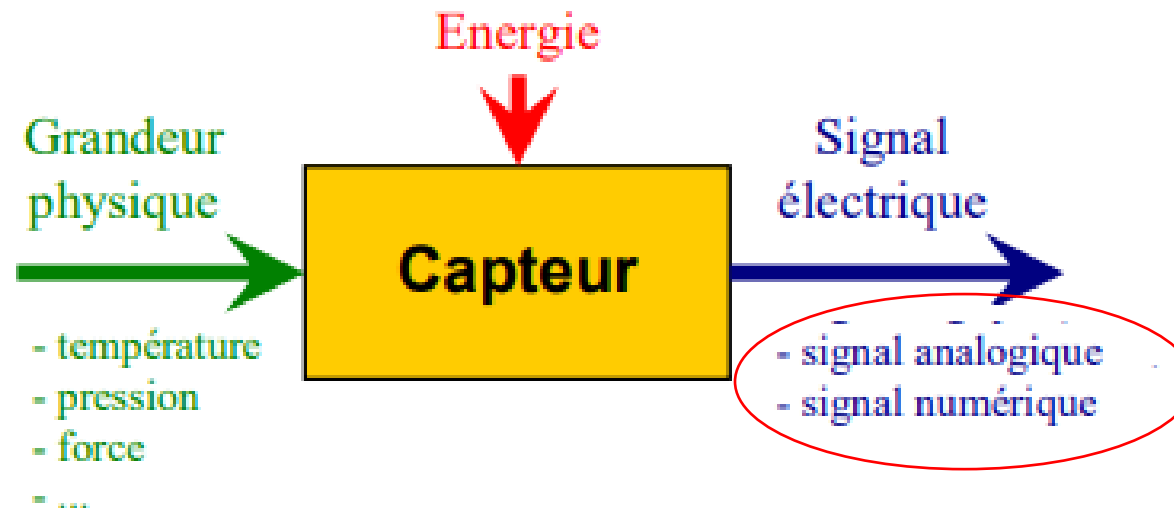
- On désire concevoir un système de régulation automatique de température d'une salle
- Si la température est supérieure à 30°C alors allumer le climatiseur
- Si la température est inférieure à 7°C alors allumer le Chauffage

Qu'est qu'il nous faut pour concevoir ce système?

- Capteur de température
- μ processeur
- Mémoire
- Chauffage
- Climatiseur

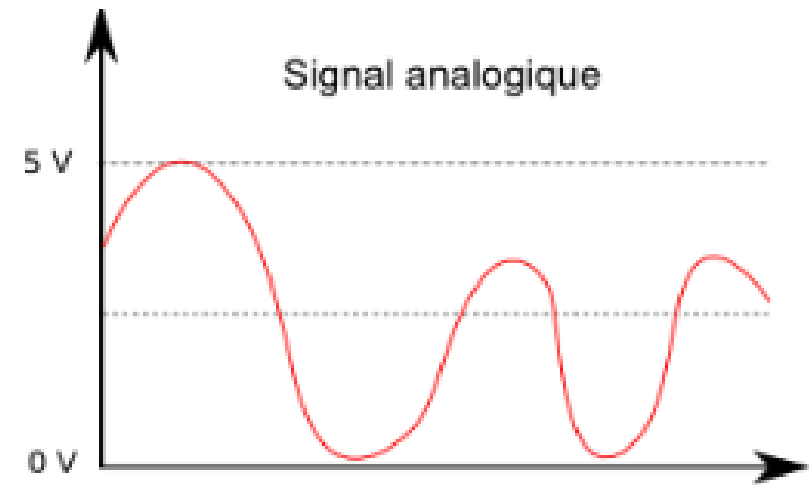
Quel est le rôle du capteur?

- Un **capteur** est un dispositif permettant de détecter un phénomène physique et le transforme sous la forme d'un signal électrique (analogique ou numérique)



Signal analogique

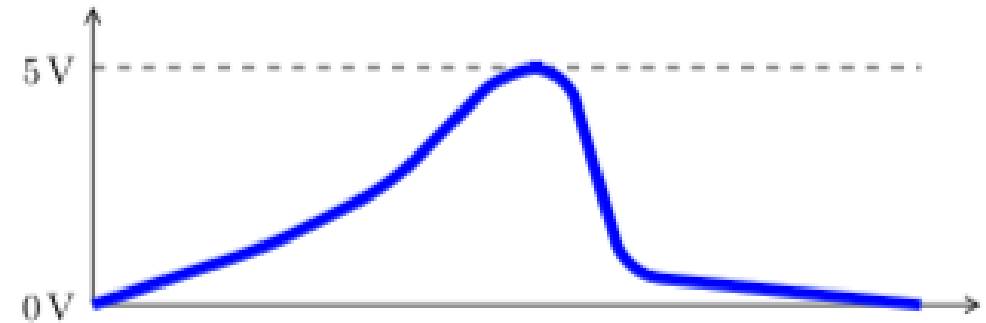
- Un signal analogique est une forme d'onde continue au cours du temps.
- Un signal analogique est celui qui représente les variations d'une grandeur physique (pression atmosphérique, la température,..)
- le capteur qui fournit ce signal est un capteur analogique



Signal analogique

Exemple: Capteur de température

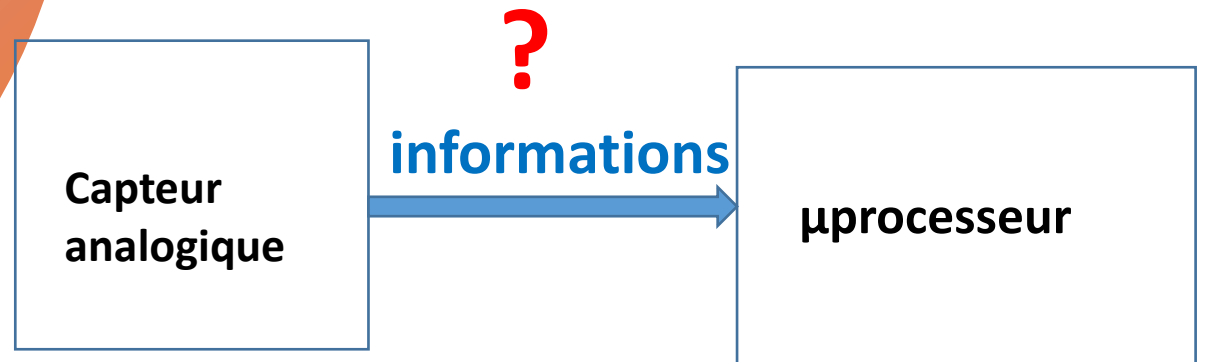
- Le Capteur de température fournit de manière continue un signal électrique, compris entre 0 V et 5 V, qui est proportionnel à une température mesurée comprise entre 0 °C et 50 °C.
- La figure suivante montre un exemple du signal électrique produit par un tel capteur au cours du temps.



Problème

?

- Sachant que le μ processeur est circuit numérique il ne traite que des informations numériques
- Comment le μ processeur puisse traiter les informations venant du capteur analogique ?
- Comment on va numériser le signal analogique?

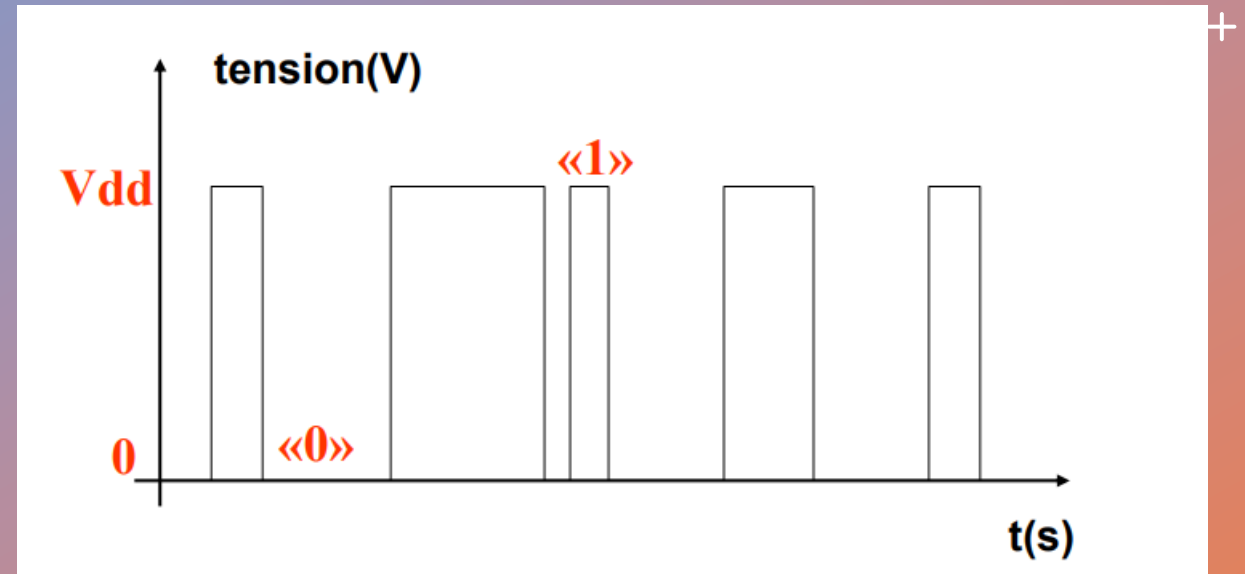


Signal numérique

Pour que le μ processeur puisse traiter les informations venant du capteur analogique il faut numériser ces informations utilisant Convertisseur Analogique Numérique CAN qui permet de convertir un signal analogique en un signal numérique pouvant être traité par une logique numérique

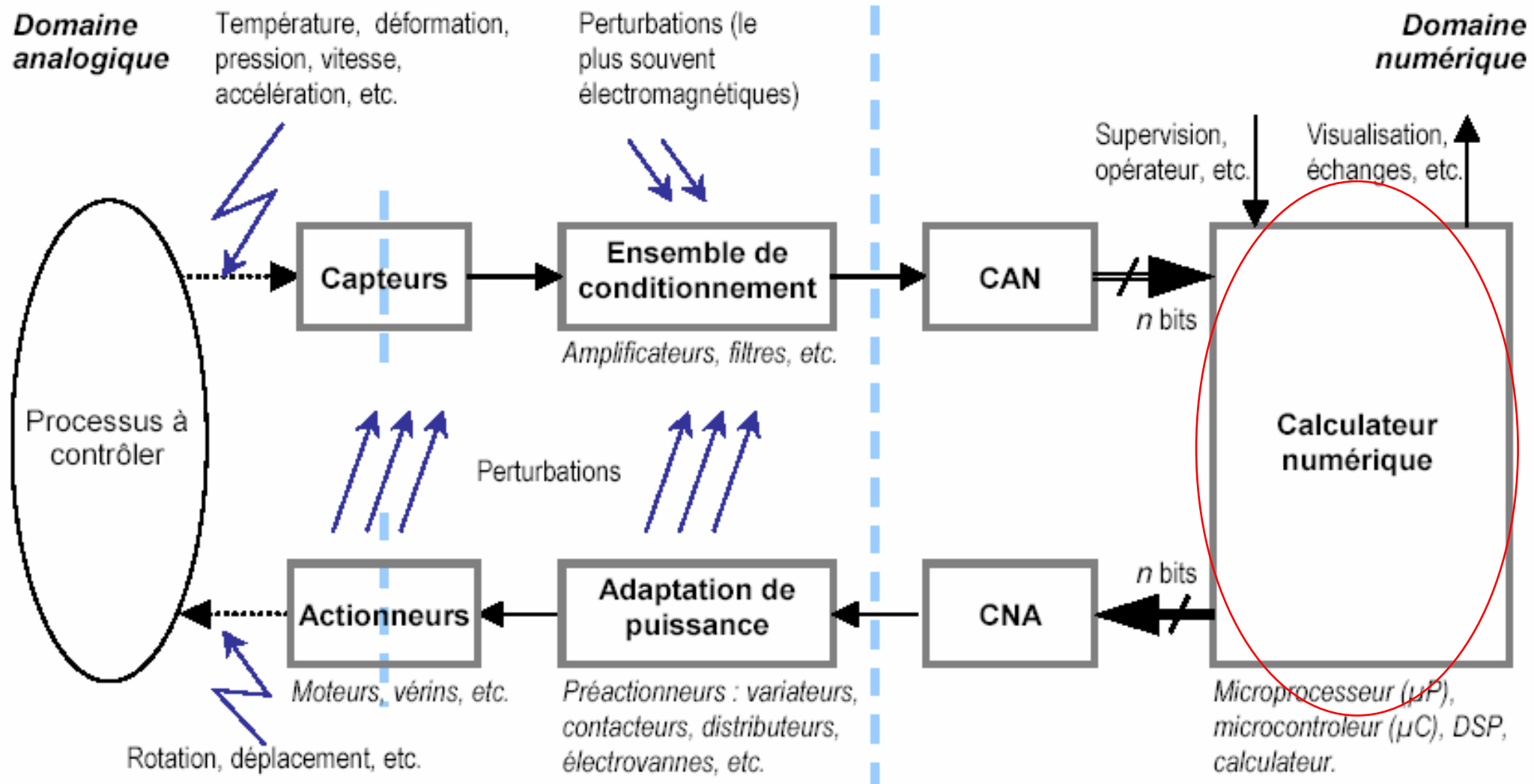


Signal numérique



Le signal n'est défini que par deux grandeurs , 0 ou 1 On dira que le signal est discret

Chaine de traitement numérique



Microprocesseur

- Le microprocesseur est un composant électronique qui effectue tous les calculs, exécute les instructions des programmes...
- Élément de base c'est le transistor (interrupteur)
- Si le transistor laisse passer le courant électrique (interrupteur fermé).
- S' il ne le laisse pas passer le courant (interrupteur ouvert).

Microprocesseur

- Un processeur peut exécuter des milliers d'instructions différentes ;
- la complexité de son architecture.
- Cette complexité se mesure par le nombre de transistors contenus dans le microprocesseur.
- Plus le microprocesseur contient de transistors, plus il pourra effectuer des opérations complexes

Représentation des nombres

- La représentation interne des données consiste à donner un codage binaire à l'information :

Information externe=> **codage**=> information interne binaire (0 ou 1)



Représentation des nombres

Décimal base 10 chiffres 0, 1, ..., 9

Binaire base 2 chiffres 0 et 1

Octal base 8 (octal) chiffres 0, ..., 7

Hexadécimal base 16 Chiffres 0,.....9, A, B, C, D, E, F

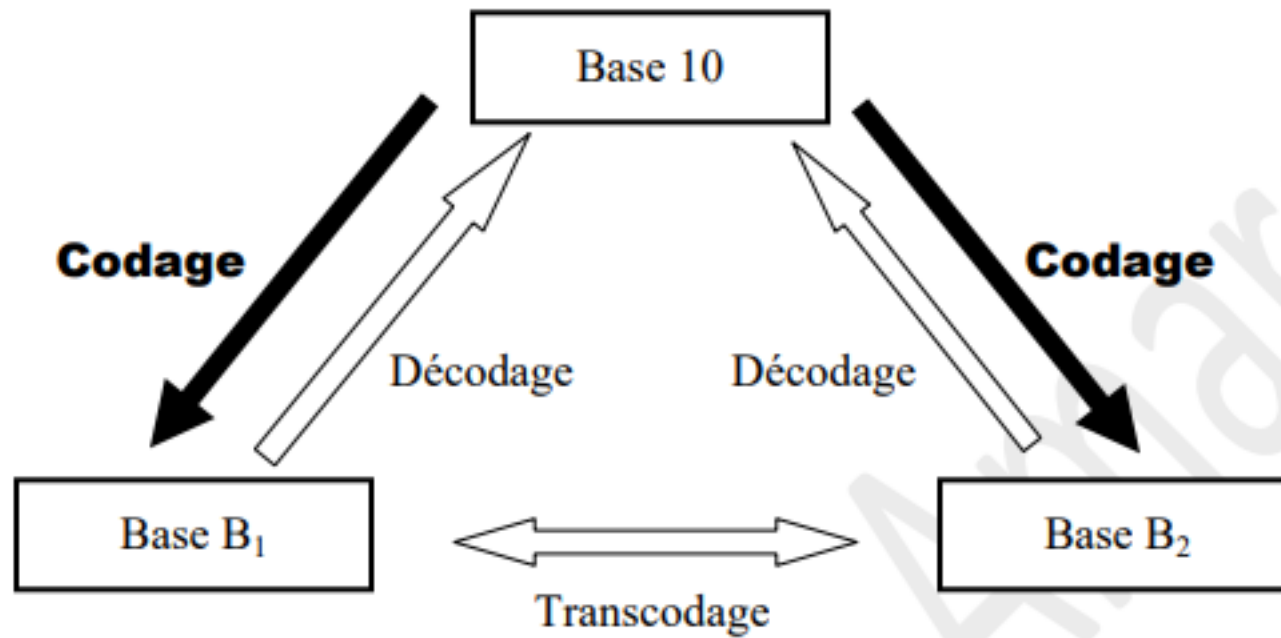
↓ ↓ ↓ ↓ ↓ ↓

10 11 12 13 14 15

Représentation des nombres Entiers non signés

- Les nombres non signés ce sont des nombres positifs.
- Conversion de base est l'opération qui permet de passer de la représentation d'un nombre N exprimé dans une base b_1 à la représentation du même nombre mais exprimé dans une autre base b_2
- On cite les conversions suivantes:
- **Codage** : il consiste à passer du Décimal vers Binaire, Octale, Hexadécimale, ...
- **Décodage** : il consiste à passer du Binaire, Octale, Hexadécimale vers Décimal
- **Transcodage** : il consiste à passer d'une base $b \neq 10$ vers une base $b \neq 10$

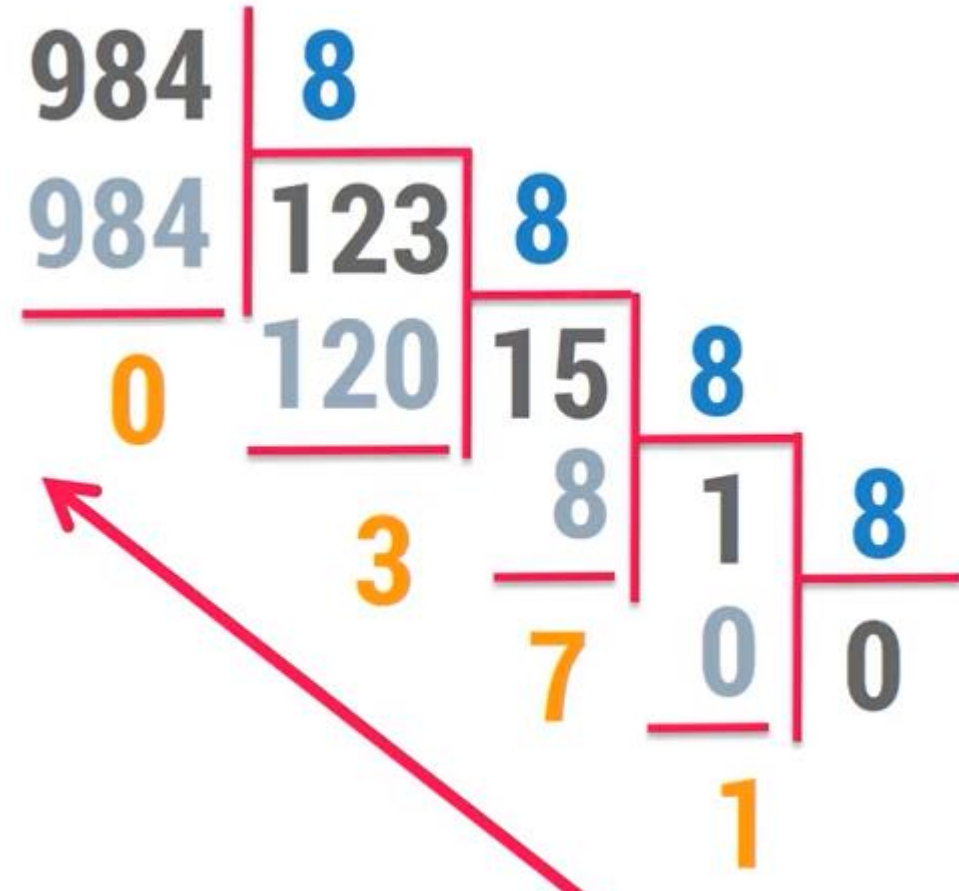
Représentation des nombres Entiers non signés



Représentation des nombres Entiers non signés

- Exemple2: Conversion décimal=> octale

$$(984)_{10} = (1730)_8$$

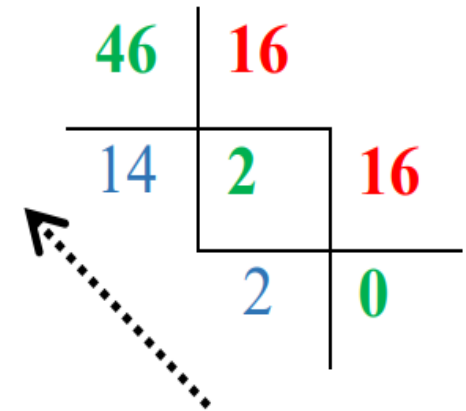


Représentation des nombres Entiers non signés

Exemple3: Conversion décimal=> hexadécimal

- Convertir le nombre $N=(46)_{10}$ en hexadécimal.

$$N = (2E)_{16}$$



Représentation des nombres Entiers non signés

□ Décodage

Le permet d'effectuer la conversion d'un nombre N de base b quelconque vers la base Décimale.

Tout nombre N peut être décomposé en fonction de puissances entières de la base. Donc on peut écrire N sous la forme polynomiale:

$$N = a_n \cdot b^n + a_{n-1} \cdot b^{n-1} + a_{n-2} \cdot b^{n-2} + \dots + a_0 \cdot b^0$$

Tel que :

n : rang du chiffre a_n

a_n : le chiffre de poids le plus fort (Most Significant Bit : MSB)

a_0 : le chiffre de poids le plus faible (Least Significant Bit : LSB).

b : La base

Représentation des nombres Entiers non signés

Exemples 1 :

Déterminer la valeur décimale du nombre $N = (213)_6$

$$(213)_6 = 2 \times 6^2 + 1 \times 6^1 + 3 \times 6^0 = 81$$

$$N = (81)_{10}$$

Représentation des nombres Entiers non signés

Conversion du nombre binaire entier $N_2 = 11010011_2$ en base 10.

$$N = 1 * 2^7 + 1 * 2^6 + 0 * 2^5 + 1 * 2^4 + 0 * 2^3 + 0 * 2^2 + 1 * 2^1 + 1 * 2^0 = 211_{10}.$$

Conversion du nombre octal entier $N_8 = 4513_8$ en base 10.

$$N = 4 * 8^3 + 5 * 8^2 + 1 * 8^1 + 3 * 8^0 = 2379_{10}.$$

Représentation des nombres Entiers non signés

□ Transcodage :

- Le Transcodage permet d'effectuer la conversion d'un nombre N de base b_1 quelconque avec ($b_1 \neq 10$) vers une autre base b_2 tel que ($b_2 \neq 10$).

Représentation des nombres Entiers non signés

□ Transcodage :

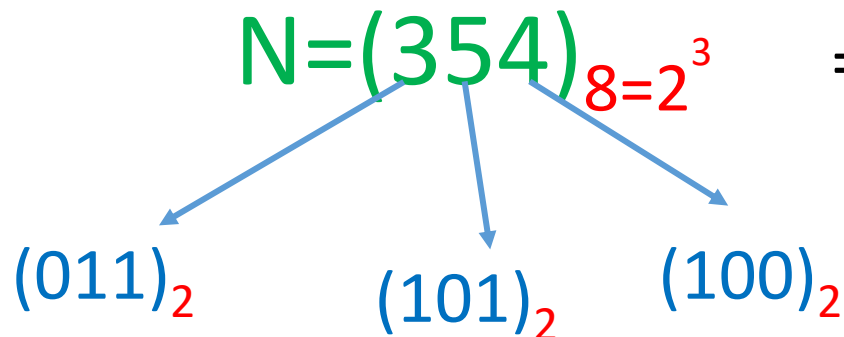
Cette méthode s'applique lorsque la base b_1 est une puissance de 2 ($b_1=2^n$) et la base b_2 correspond à 2.

Chaque chiffre de nombre N sera exprimé sous la forme d'un nombre binaire ayant n bits.

Exemple 1: octale=>binaire

$$N=(354)_8=(?)_2$$

=> éclatement sur 3 bits



$$N = (354)_8 = (011 \ 101 \ 100)_2$$

Représentation des nombres Entiers non signés

□ Transcodage : hexadécimal => binaire

Exemple2 :

$$(7D6)_{16} = (0111\ 1101\ 0110)_2$$

Représentation des nombres Entiers non signés

❑ Transcodage : binaire => hexadécimale

16=2⁴ → regroupement de 4 bits

$$N2 = (011101100110)_2 = (\underbrace{0111}_7 \underbrace{0110}_6 \underbrace{0110}_6)_2 = (766)_{16}$$

❑ Transcodage : binaire => octale

$$(11001010010110)_2 = (\underline{011} \underline{001} \underline{010} \underline{010} \underline{110})_2 = (31226)_8$$

Représentation des entiers non signés

Opérations arithmétiques en binaire

Addition

$0+0=0$; $1+0=0+1=1$; $1+1=0$ avec une retenue égale a 1 ; $1+1+1= 1$ avec une retenue égale à 1

Effectuer l'addition suivante en binaire $(11011)_2 + (100110)_2$

nue 1 1
 + $\begin{array}{r} 1\ 1\ 0\ 1\ 1\ 1 \\ 1\ 0\ 0\ 1\ 1\ 0 \\ \hline 1\ 0\ 1\ 1\ 1\ 0\ 1 \end{array}$

$$(110111)_2 + (100110)_2 = (1011101)_2$$

Représentation des entiers non signés

Opérations arithmétiques en binaire

Soustraction

$0-0=0$; $1-0=1$; $1-1=0$; $0-1 = 1$ avec '1' emprunté

Effectuer la soustraction suivante en binaire $(1100111)_2 - (101100)_2$

$$\begin{array}{r} 1\ 1\ 1\ 0\ 1\ 1\ 1 \\ \underline{1\ 1\ 1\ 0\ 1\ 0\ 0} \\ =\ 0\ 1\ 1\ 1\ 0\ 1\ 1 \end{array}$$

$$(1100111)_2 - (101100)_2 = (0111011)_2$$

Représentation des entiers non signés

Opérations arithmétiques en binaire

Effectuer la multiplication suivante en binaire $(111101)_2 \times (101)_2$

Retenue

$$\begin{array}{r} 111101 \\ \times 101 \\ \hline 111 \\ + 1111101 \\ + 1000000. \\ + 111101.. \\ \hline = 100110001 \end{array}$$

$$(111101)_2 \times (101)_2 = (100110001)_2$$

Représentation des entiers non signés

Opérations arithmétiques en binaire

- Si le dividende est supérieur ou égale au diviseur alors le quotient est égal à 1 sinon il est égal à 0

Dividende

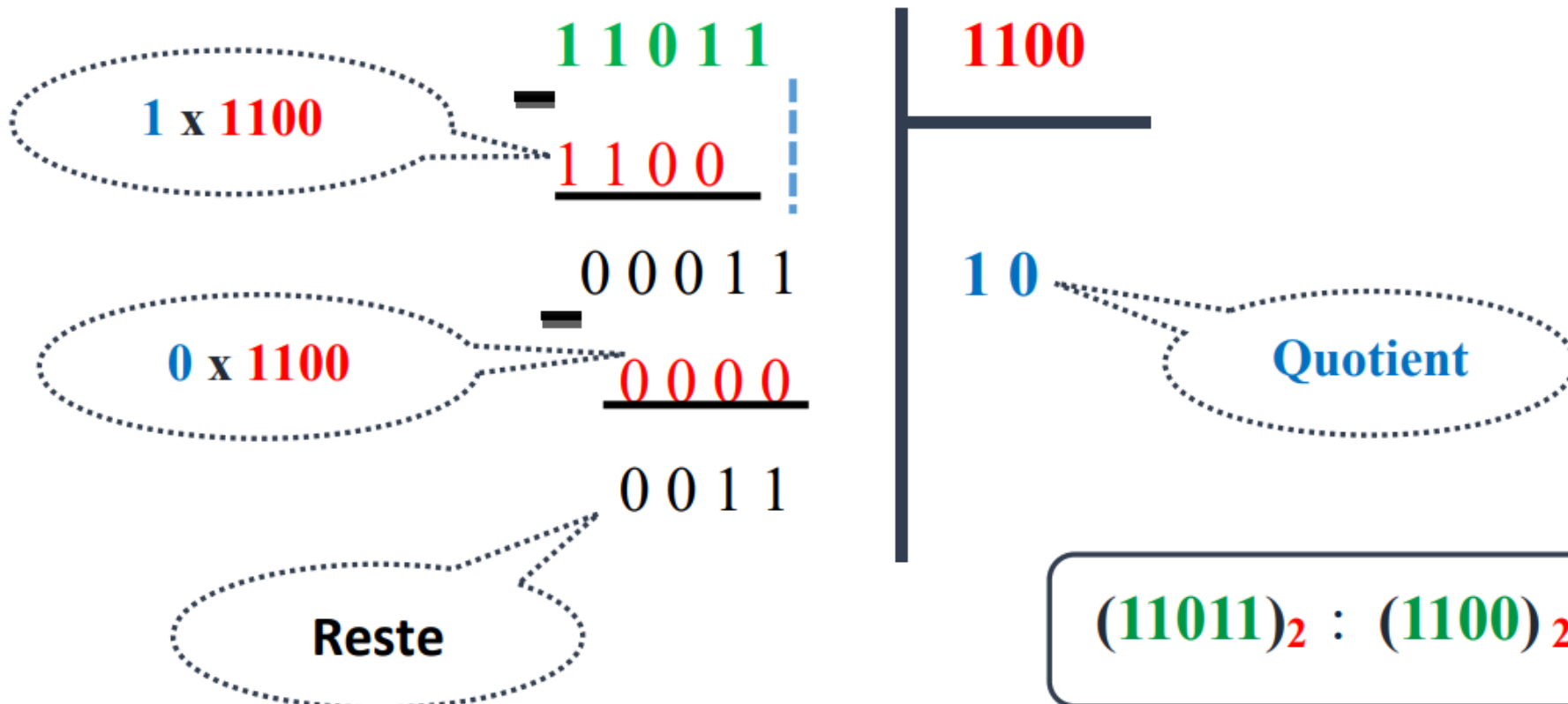
Diviseur

Quotient.

Représentation des entiers non signés

Opérations arithmétiques en binaire

Effectuer la division suivante en binaire $(11011)_2 : (1100)_2$



Représentation des entiers signés

- Dans la représentation des nombres binaires signés (positifs ou négatifs) sont représentés uniquement en 0 et 1.
- Le bit le plus significatif indique le signe du nombre 0 pour positif et 1 pour négatif

Représentation des entiers signés

- Il existe deux types d'entiers :
 - les entiers non signés (positifs)
 - les entiers signés (positifs ou négatifs)
- Il existe 3 méthodes pour représenter les nombres négatifs :
 - Signe valeur absolue (SVA)
 - Complément à 1 (C1)
 - Complément à 2 (C2)

Représentation des entiers signés

Signe et valeur absolue (SVA)

- Si un nombre est représenté sur n bits, alors la valeur absolue du nombre est codée sur $(n-1)$ bits et le signe est codé sur le bit du poids fort, par convention **0** représente un nombre **positif** et **1** un nombre **négatif** tels que :
- L'intervalle des nombres représentables sur n bits en SVA est :
$$[-(2^{n-1}-1), + (2^{n-1}-1)]$$
- Le nombre de représentations possibles sur n bits est 2^n .

Représentation des entiers signés

Signe et valeur absolue (SVA)

Pour la représentation signe et valeur absolue **SVA** sur **4bits** de **+7** et **-7**

Non signé

$$7 = (111)_2$$



Signé

+7 s'écrit **(0111)_{SVA}**

-7 s'écrit **(1111)_{SVA}**

Représentation des entiers signés

Signe et valeur absolue (SVA)

Sur **n** bits , l'intervalle des valeurs qu'on peut représenter en système en valeur absolue:

$$-(2^{(n-1)} - 1) \leq N \leq +(2^{(n-1)} - 1)$$

signe	VA	valeur
0	00	+ 0
0	01	+ 1
0	10	+ 2
0	11	+ 3
1	00	- 0
1	01	- 1
1	10	- 2
1	11	- 3

Exemple: pour n=3 bits

$$-(2^{(3-1)} - 1) \leq N \leq +(2^{(3-1)} - 1)$$

Représentation des entiers signés

Complément à 1 (C1)

- Dans cette représentation : le premier bit est réservé pour le signe
- Si le nombre est positif alors il garde son format
- Si le nombre est négatif alors chaque bit est inversé (0 devient 1 et 1 devient 0).
- L'intervalle des nombres représentables sur n bits en C1 est :

$$[-(2^{n-1}-1), + (2^{n-1}-1)]$$

- Le nombre de représentations possibles sur n bits est 2^n .

Représentation des entiers signés

Complément à 1 (C1)

Exemple:

le complément à 1 (**CP1**) sur **4bits** de **+7** et **-7** :

$$7 = (111)_2$$

$$7 \text{ sur } 4\text{bits} = 0111$$

-7 s'écrit $\begin{array}{c} \text{1111} \\ (1000)_{\text{CP1}} \end{array}$ (Signe - on **inverse**)

+7 s'écrit $(0111)_{\text{CP1}}$ (Signe + on **n'inverse pas**)

Représentation des entiers signés

Complément à 1 (C1)

Sur n bits , l'intervalle des valeurs qu'on peut représenter en CA1 :

$$-(2^{(n-1)} - 1) \leq N \leq +(2^{(n-1)} - 1)$$

Exemple : sur 3 bits, les valeurs sont comprises entre -3 et +3

$$-(2^{(3-1)} - 1) \leq N \leq +(2^{(3-1)} - 1)$$

Représentation des entiers signés

Complément à 1 (C1)

Sur 3 bits :

Valeur en CA1	Valeur décimal
000	+ 0
001	+ 1
010	+ 2
011	+ 3
100	- 3
101	- 2
110	- 1
111	- 0

Dans cette représentation , le bit du poids fort nous indique le signe:
0 : positif , 1 : négatif).

Représentation des entiers signés

Complément à 1 (C1)

- *L'addition de deux nombres de même signe* peut donner lieu à un **dépassement** de capacité.
- Cas de deux entiers de signe positif.
- On a un dépassement de capacité quand le bit de signe du résultat vaut 1.

sur 8 bits on code les nombres de $[-127, +127]$

Exemple

+103	:	0110	0111
+65	:	0100	0001
+168	≠	1010	1000

on obtient un nombre **négalif** qui représente -87 en complément à un.

=> le résultat est faux sur 8 bits

=> **dépassement de capacité**

=> il faut coder sur 9 bits 42

Représentation des entiers signés

Complément à 1 (C1)

- Cas de deux entiers de signe négatif

Exemple1

-35	:	1101	1100
-65	:	1011	1110
		1	1001 1010
-100	:	1001	1011

=> Pas de dépassement de capacité

=> Pour déterminer l'équivalent en décimale il faut ajouter 1 puis faire le C1 (1001 1010)

Exemple2

-103	:	1001	1000
-65	:	1011	1110
		1	0101 0110
-168	≠	0101	1000

=> Le résultat est faux sur 8 bits

=> **Dépassement de capacité**

=> il faut coder sur 9 bits

Représentation des entiers signés

Complément à 1 (C1)

- *L'addition de deux nombres de signes opposés*
 - pas de dépassement de capacité ;
 - s'il n'y a pas de retenue, le résultat est un nombre négatif dont la valeur absolue est obtenue en inversant les bits ;
 - s'il y a une retenue, le résultat est un nombre positif, on ajoute la retenue pour avoir sa valeur ;

Représentation des entiers signés

Complément à 1 (C1)

- Exemple 1:

$$-13+4=$$

$$(13)_{10}=(01101)_2$$

$$(-13)_{10}=(10010)_{C1}$$

$$(4)_{10}=(00100)_{C1}$$

$$(-13)_{10}+(4)_{10}=$$

$$(10010)_{C1}$$

$$+(00100)_{C1}$$

$$=(\textcolor{red}{1}0110)_{C1}=(-9)_{10}$$

On remarque qu'il n'y a pas de retenue et le résultat est un nombre **néga**tif en C1

Représentation des entiers signés

Complément à 1 (C1)

- **Exemple 2:** Effectuer $(6-4)$ sur 4bits en utilisant le complément à 1

$$(6)_{10} = (0110)_2$$

$$(4)_{10} = (0100)_2$$

$$(-4)_{10} = (1011)_{C1}$$

$$(-4)_{10} + (6)_{10} =$$

$$(1011)_{C1}$$

$$+(0110)$$

$$=(10001)_{C1}$$

$$+ \begin{array}{c} \downarrow \\ 1 \end{array}$$

$$=(0010)_2 = (2)_{10}$$

On remarque qu'il y a une retenue et le résultat est un nombre positif, on ajoute la retenue pour avoir sa valeur.

Représentation des entiers signés

Complément à 2 (C2)

- le premier bit est réservé pour le signe, et si le nombre est positif alors il garde son format,
- Si le nombre est négatif il est transformé en C1 puis on ajoute 1.
- Le nombre de représentations possibles sur n bits est 2^n .
- l'intervalle des nombres représentables sur n bits en C2 est :

$$[-2^{n-1}, + (2^{n-1}-1)]$$

Sur 4 bits :

	SVA		CP1		CP2	
+N	+7	0111	+7	0111	+7	0111
	+6	0110	+6	0110	+6	0110
	+5	0101	+5	0101	+5	0101
	+4	0100	+4	0100	+4	0100
	+3	0011	+3	0011	+3	0011
	+2	0010	+2	0010	+2	0010
	+1	0001	+1	0001	+1	0001
	+0	0000	+0	0000	+0	0000
-N	-0	1000	-0	1111	-1	1111
	-1	1001	-1	1110	-2	1110
	-2	1010	-2	1101	-3	1101
	-3	1011	-3	1100	-4	1100
	-4	1100	-4	1011	-5	1011
	-5	1101	-5	1010	-6	1010
	-6	1110	-6	1001	-7	1001
	-7	1111	-7	1000	-8	1000

Complément à 2 (C2)

Addition en complément à 2

Elle se base sur le principe suivant :

- S'il y a une retenue, elle est **ignorée**
- Sinon le résultat est correct et il est représenté en C2.
- Exemple1 :

$$\begin{aligned} &+13-4=+13+(-4) \\ &=(01101)_{C2}+(11100)_{C2} \\ &=(\textcolor{green}{1}\textcolor{red}{0}1001)_{C2} \end{aligned}$$

une retenue générée sera négligée

Bit de signe 0 le résultat est positif $=(01001)_2=(+9)_{10}$

Complément à 2 (C2)

Addition en complément à 2

- Exemple 2 :

$$-13+4$$

$$=(10011)_{C2}+(00100)_{C2}$$

$$=(\textcolor{red}{1}0111)_{C2}$$

On remarque qu'il n'y a pas de retenue, le résultat est un nombre **négatif** en C2 pour le transformer en forme naturelle il faut appliquer de nouveau le C2,

$$C2((\textcolor{red}{1}0111)_{C2})=(01001)_2=(-9)_{10}$$

Complément à 2 (C2)

Addition en complément à 2/ débordement

- Il y a risque de débordement (overflow) lorsque la somme d'une addition dépasse le nombre maximal permis.
- Pour un système à n bits, il y a débordement si la somme est plus grande que $+(2^{n-1} - 1)$, ou si la somme est plus petite que $-(2^{n-1})$.
- Si on additionne deux nombres de signe différent, il n'y a jamais de débordement.
- Cependant, si on additionne deux nombres de même signe, il y a possibilité de débordement.
- Il y a alors dépassement de capacité lorsque les deux opérandes sont de même signe et que le résultat est de signe opposé

Complément à 2 (C2)

Addition en complément à 2/ débordement

- Une retenue sur un bit supplémentaire par rapport à la précision ne veut pas forcément dire que le résultat n'est pas stockable avec la précision utilisée
- On regarde les retenues des deux derniers bits (poids forts) additionnés pour savoir s'il y a eu débordement
- Si retenues identiques (00 ou 11) : pas de débordement
- Si retenues différentes (01 ou 10) : débordement
- S'il n'y a pas de débordement, le résultat tient dans la précision requise, la retenue n'a aucune signification

Complément à 2 (C2)

Addition en complément à 2/ débordement

- Exemples de calcul avec codage des entiers signés en complément à 2, précision de 5 bits

$$9 = (01001)_2 \quad 8 = (01000)_2 \quad 5 = (00101)_2$$

$$-9 = (10111)_2 \quad -8 = (11000)_2 \quad -5 = (11011)_2$$

Complément à 2 (C2)

Addition en complément à 2/ débordement

$$\begin{array}{r} \textcircled{11} \\ 01001 = 9 \\ + 11000 = -8 \\ \hline \textcircled{1}00001 \end{array}$$

Le 5^{ème} bit = 0 : nombre positif
Résultat = $(00001)_2 = 1$

Calcul correct

Pas de dépassement de capacité

la retenue n'a aucune signification

Complément à 2 (C2)

Addition en complément à 2/ débordement

$$\begin{array}{r} \textcircled{00} \\ 00101 = 5 \\ + 11000 = -8 \\ \hline 0\underline{1}1101 \end{array}$$

Calcul correct

Pas de dépassement de capacité

Complément à 2 (C2)

Addition en complément à 2/ débordement

Calcul de 9 + 8 :

$$\begin{array}{r} \textcircled{01} \quad \quad \quad \rightarrow \text{retenues} \\ 01001 = 9 \\ + 01000 = 8 \\ \hline 010001 \end{array}$$

Résultat tient sur 5 bits mais calcul faux

dépassement de capacité

5 bits insuffisant pour coder (9+8)

=>il faut utiliser 6 bits

Complément à 2 (C2)

Addition en complément à 2/ débordement

$$\begin{array}{r} \textcircled{10} \\ 10111 = -9 \\ + 11000 = -8 \\ \hline 101111 \end{array}$$

Calcul incorrect

dépassement de capacité
5 bits insuffisant pour coder (-9-8)
=>il faut utiliser 6 bits => prendre la retenue

Complément à 2 (C2)

Addition en complément à 2/ débordement

$$\begin{array}{r} \textcircled{11} \\ 11011 = -5 \\ + 11000 = -8 \\ \hline 110011 \end{array}$$

Calcul correct

Pas de dépassement de capacité

Ignorer la retenue

Calcul correct car 2 dernières retenues sont identiques

Le 5^{ème} bit = 1 : nombre négatif

Valeur absolue du résultat = $\text{comp}_2(10011) = (01101)_2 = 13$

On ignore systématiquement le 6^{ème} bit

Donc résultat de $-5 - 8 = -13$

Représentation des nombres Fractionnaires non signés

□ Codage : décimal=> binaire

- Nombre réel est constitué de deux parties : la partie entière et la partie fractionnaire.
- La partie entière: est transformée en effectuant des divisions successives.
- La partie fractionnaire: est transformée en effectuant des multiplications successives par 2 .
- Exemple : effectuer la conversion suivante: $(37,625)_{10}$

Partie entière = $37 = (100101)_2$

Partie fractionnelle = $0,625 = (?)_2$

$$(0,625) = (0,101)_2$$

$$\text{Donc } 37,625 = (100101,101)_2$$

Partie fractionnelle

$$0,625 * 2 = \boxed{1},25$$

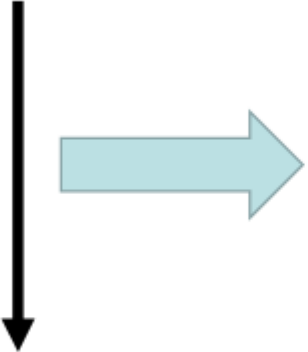
$$0,25 * 2 = \boxed{0},5$$

$$0,5 * 2 = \boxed{1},0$$



Représentation des nombres Fractionnaires non signés

Exemple 2: Effectuer la conversion suivante $(0,7)_{10} = (?)_2$

$0,7 * 2 = 1,4$		$(0,7) = (0,10110)_2$
$0,4 * 2 = 0,8$		
$0,8 * 2 = 1,6$		
$0,6 * 2 = 1,2$		
$0,2 * 2 = 0,4$		

Le nombre de bits après la virgule va déterminer la précision

Représentation des nombres Fractionnaires non signés

❑ Codage : décimal=> octal

Soit à convertir en **octal** ($b=8$) le nombre décimal $N = (0,732)_{10}$

- **Partie Entière:** $(0)_{10} = (0)_8$
- **Partie Fractionnaire:** $(0,732)_{10}$

$$\begin{array}{l} 0,732 \times 8 = 5,856 \\ 0,856 \times 8 = 6,848 \\ 0,848 \times 8 = 6,784 \\ 0,784 \times 8 = 6,272 \\ 0,272 \times 8 = 2,176 \end{array}$$




$$N = (0,732)_{10} = (0,56662)_8$$

Représentation des nombres Fractionnaires non signés

❑ **Décodage:** Pour réaliser le Décodage d'un nombre fractionnaire N de base b quelconque vers la base 10, on effectue les opérations de l'expression de la forme polynomiale de N.

Soit à déterminer la valeur décimale du nombre N

$$\blacksquare \quad N = (11,101)_2 = (1 \times 2^1 + 1 \times 2^0, 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3})_{10}$$

$$N = (3,625)_{10}$$

$$\blacksquare \quad N = (20,41)_8 = (2 \times 8^1 + 0 \times 8^0 + 4 \times 8^{-1} + 1 \times 8^{-2})_{10}$$

$$N = (16,515625)_{10}$$

Représentation des nombres Fractionnaires non signés

□ Transcodage :

1°/ A partir de la virgule, grouper les bits par groupes de n en allant vers la gauche pour la partie entière et vers la droite pour la partie fractionnaire.

2°/ Convertir ensuite chaque bloc séparément en base b selon le code binaire naturel.

Représentation des nombres Fractionnaires non signés

↔
Partie Entière , Partie Fractionnaire

Exemple 1 :

- Binaire en octal

Soit à convertir en octal le nombre binaire $N = (111011101,000100)_2$

$8=2^3 \rightarrow$ regroupement de 3 bits

$$N = (111011101,000100)_2 = (\underbrace{111}_7 \underbrace{011}_3 \underbrace{101}_5, \underbrace{000}_0 \underbrace{100}_4)_2 = (735,04)_8$$

Représentation des nombres Fractionnaires non signés

- **Conversion : binaire -> octal** le regroupement se fait de droit à gauche pour la partie entière et de gauche à droite pour la partie fractionnelle .

$$(110010100, 10101)_2 = (\underline{110} \ \underline{010} \ \underline{100}, \ \underline{101} \ \underline{010})_2 = (624, 52)_8$$

- **Conversion : binaire -> hexadécimal**

$$(110000101, 10111)_2 = (\underline{0001} \ \underline{1000} \ \underline{0101}, \ \underline{1011} \ \underline{1000})_2 = (185, B8)_{16}$$

- **Conversion : hexadécimal -> binaire**

$$(BA3, 5F7)_{16} = (\underline{1011} \ \underline{1010} \ \underline{0011}, \ \underline{0101} \ \underline{1111} \ \underline{0111})_2$$

Représentation des nombres réels

Format virgule fixe

Utilisé par les premières machines, possède une partie entière et une partie décimale ou fractionnaire séparées par une virgule. La position de la virgule est fixe.

Exemple : $(54,25)_{(10)}$; $(10,001)_{(2)}$; $(A1,F0B)_{(16)}$

Représentation des nombres réels

Format virgule flottante

- Exemple :

$$(3,25)_{10} = (32,5 \times 10^{-1})_{10} = (325 \times 10^{-2})_{10} = (0,325 \times 10^1)_{10}$$

La conversion de 3,25 en binaire donne :

$$0,25 \times 2 = \underline{0},5$$

$$0,5 \times 2 = \underline{1},0$$

$$\begin{array}{r|l} 3 & 2 \\ \hline \underline{1} & 1 \\ & \underline{1} \end{array} \quad \begin{array}{r|l} & 2 \\ \hline & 0 \end{array}$$

Donc $(3,25)_{10} = (11,01)_2$ (en virgule fixe)

Représentation des nombres réels

Format virgule flottante

- $(3,25)_{10} = (11,01)_2 = (1,101 \times 2^1)_2 = (0,1101 \times 2^2)_2 = (110,1 \times 2^{-1})_2$
(en virgule flottante)

Représentation des nombres réels

Codage en Virgule Flottante Normalisé

- Exemple

Normalisation:

$$11,01 = 1,101 \times 2^1 ; \text{ donc } M = 101$$

$$X = \pm 1, M \cdot 2^E$$

Représentation des nombres réels

Codage en Virgule Flottante Normalisé

SM	Eb	M
1bit	p bits	q bits

$$X = \pm 1, M \cdot 2^E$$

- SM : signe de la mantisse, il est codé sur 1 bit ayant le poids fort :
le signe – : bit 1 ; le signe + : bit 0
- Eb : Exposant biaisé, codé sur p bits et biaisé pour assurer d'être positif

$$\text{Exposant Biaisé} = \text{Exposant réel} + \text{Biais } (2^{p-1}-1)$$

- M : Mantisse normalisée: la virgule est placée après le bit à 1 ayant le poids fort; elle est codée sur q bits

Codage en Virgule Flottante Normalisé

Le standard IEEE 754 (1985)

□ Simple précision sur 32 bits :

- 1 bit de signe de la mantisse
- 8 bits pour l'exposant
- 23 bits pour la mantisse

Signe	Eb	M
1bit	8 bits	23 bits

Codage en Virgule Flottante Normalisé

Le standard IEEE 754 (1985)

□ Double précision sur 64 bits

- 1 bit de signe de la mantisse
- 11 bits pour l'exposant
- 52 bits pour la mantisse

SM	Eb	M
1bit	11 bits	52 bits

- Exemple1 : $(35,5)_{10} = (?)_{IEEE\ 754\ simple\ précision}$

- $(35,5)_{10} = (100011,1)_2$ (virgule fixe)

$= (1,000111 \times 2^5)_2$ (virgule flottante)

$E_b = E + 2^{p-1} - 1 = E + 2^{8-1} - 1 = E + 127 = 5 + 127 = 132 = (10000100)_2$

$1, M = 1,000111 \Rightarrow M = 000111000...$

$SM = 0$; c'est un nombre positif

0	10000100	000111 00 00 00 00 00 00 00 00 00 0
---	----------	-------------------------------------

1bit

8 bits

23 bits

- Exemple2 :
- (0**10000001**111000000000000000000000) IEEE754 simple précision=(X)₁₀
- SM=0 donc c'est nombre positif
- Eb=**10000001**=129
- Eb=E+2⁸⁻¹-1=E+127=**129**

$$\Rightarrow \text{E} = \text{Eb} - 127 = 129 - 127 = 2$$

$$\text{M} = 111000000000000000000000 = 111$$

Donc X=(1,111x2²)en virgule flottante

=(111,1)en virgule fixe

=(7,5)₁₀

Codage en Virgule Flottante Normalisé

Le standard IEEE 754 (1985)

- Addition

Elle se fait en trois étapes :

- Dénormaliser les deux nombres afin d'avoir le même exposant (le plus élevé)
- Additionner les mantisses
- Normaliser le résultat

Exemple: $A = (0 \text{ 10000001 } 110000000000000000000000)_{\text{IEEE754}}$

$B = (0 \text{ 10000000 } 001000000000000000000000)_{\text{IEEE754}}$

Dénormalisation :

Dénormalisation A

$SM=0 \Rightarrow \text{signe+}$

$Eb = 10000001 = 129$

$E = Eb - (2^{8-1} - 1) = 129 - 127 = 2$

$M = 11$

Donc $A = +1,11 \times 2^2$

Dénormalisation B

$SM=0 \Rightarrow \text{signe+}$

$Eb = 10000000 = 128$

$E = Eb - (2^{8-1} - 1) = 128 - 127 = 1$

$M = 001$

Donc $B = +1,001 \times 2^1$
 $= +0,1001 \times 2^2$

2-Addition des mantisses:

$$A+B=10,0101 \times 2^2$$

$$\begin{array}{r} 1,11 \\ +0,1001 \\ \hline 10,0101 \end{array}$$

3-Normalisation:

$$A+B=10,0101 \times 2^2$$

$$=1,00101 \times 2^3$$

$$E=3 \rightarrow E_b=E+127=3+127=130=(10000010)_2$$

$$M=00101$$

Donc $A+B=(0 \ 10000010 \ 00101 \ 00 \ 00 \ 00 \ 00 \ 00 \ 00 \ 00 \ 00 \ 00 \ 00 \ 00)_{IEEE754}$

Codage en Virgule Flottante Normalisé

Le standard IEEE 754 (1985)

- Multiplication

Elle se fait en quatre étapes :

- Dénormaliser les deux nombres (exposants naturels)
- Additionner les exposants naturels
- Multiplier les mantisses
- Normaliser le résultat

Exemple $7 * 9,5 = ?$

$(7)_{10} = (111)_2 = (0\ 10000001\ 110000000000000000000000)_{\text{IEEE754}}$

$(9,5)_{10} = (1001,1)_2 = (0\ 10000010\ 001100000000000000000000)_{\text{IEEE754}}$

- Dénormalisation :

(0 10000001 110000000000000000000000)IEEE754=1,11*2²

(0 10000010 001100000000000000000000)IEEE754=1,0011*2³

- Addition des exposants : E=2+3=5

- Multiplication des mantisses : 1,11*1,0011=10,000101

- Normalisation

10,000101*2⁵=1,0000101*2⁶

SM=0 ;

E=6=>Eb=6+127=133=(10000101)₂ ;

M=0000101

Donc le résultat est =(0 10000101 000010100000000000000000)IEEE754